Lectures on Monte Carlo Theory

Chapter 1: Introduction

Based on Leskelä & Vihola

Chapter Outline

What is Monte Carlo (MC)?

Monte Carlo methods are computational algorithms that use *random (or pseudo-random) numbers* to solve mathematical and statistical problems: numerical integration, optimisation, simulation of stochastic systems, and statistical inference.

Historical origin: Developed in the 1940s at Los Alamos by von Neumann, Ulam, and Metropolis; named after the Monte Carlo casino.

Why randomness helps:

- High-dimensional integrals — grid methods cost n^d (exponential in d).
- NP-hard combinatorial problems (TSP, knapsack).
- Complex stochastic systems without closed-form solutions.
- Bayesian posterior distributions.

1.1 About the Book II

Book contents:

1. **Ch.1** Introduction: motivating examples
2. **Ch.2** Theory of random number generators and statistical tests
3. **Ch.3** Generating random variables
4. **Ch.4** Output analysis: confidence intervals, estimators
5. **Ch.5** Variance reduction (stratification, antithetic, IS, CE)
6. **Ch.6** Markov Chain MC (Metropolis, Gibbs, CFTP)
7. **Ch.7** Stochastic optimisation (SA, cross-entropy)
8. **Ch.8** Simulation of queues and related processes

Key distinction: MC vs Quasi-MC

Monte Carlo uses *pseudo-random* sequences (deterministic but statistically random-looking).
Quasi-Monte Carlo (QMC) uses carefully designed *low-discrepancy* sequences that fill $[0,1]^d$ more evenly.

1.1 Pseudorandom Numbers in Python

NumPy recommended API

Use `default_rng` with an explicit seed for reproducibility.

Listing 1: Initialising a PRNG (Book, Sec. 1.2 intro)

```python
import numpy as np
from numpy.random import PCG64, default_rng

# Recommended: default_rng with explicit seed
rng = np.random.default_rng(seed=1234)

# Generate 10 uniform samples on [0, 1)
u = rng.random(10)

# Underlying generator can be specified
rng2 = default_rng(PCG64(seed=1234))

# Successive calls produce statistically i.i.d.-looking numbers
u1 = rng2.random(5)
u2 = rng2.random(5)    # different values, same stream
```

Successive pseudorandom numbers mimic i.i.d. Uniform$[0, 1]$ *random variables, but are entirely deterministic given the seed.*

1.2 Monte Carlo: Simple Probabilistic Tasks I

Section overview

Three motivating examples, each easy to *simulate* but hard (or impossible) to solve analytically at scale:

1. Simple Symmetric Random Walk (SSRW)
2. Travelling Salesman Problem (TSP)
3. Computing multidimensional integrals

Python generator for these examples (Listing 1.1 in book):

```
rng = np.random.default_rng(PCG64(seed=1234))
```

The PCG64 generator is the default in NumPy ≥ 1.17; it passes all standard statistical tests and has a period of 2^{128}.

1.2.1 Simple Symmetric Random Walk: Setup I

Motivating game (Feller, Ch. III): Players A and B bet on coin tosses. A wins $+1$ on heads, -1 on tails. Starting fortune $S_0 = 0$; consider $2N$ tosses.

Definition – Simple Symmetric Random Walk (SSRW)

Let $X_1, X_2, \ldots$ be i.i.d. with $\mathbb{P}(X_j = +1) = \mathbb{P}(X_j = -1) = \frac{1}{2}$.

$$S_0 = 0, \qquad S_n = \sum_{j=1}^{n} X_j, \quad n \geq 1.$$

The sequence $S_0, S_1, \ldots$ is the *simple symmetric random walk*.

Geometric interpretation: A line segment connects $(i-1, S_{i-1})$ to (i, S_i). A segment lies *above* the x-axis if $S_{i-1} \geq 0$ or $S_i \geq 0$.

1.2.1 Simple Symmetric Random Walk: Setup II

Four questions about the realization $(S_n)_{n=0,\dots,2N}$:

- Q1 **Last tie:** Let $L_{2N} = \max\{i : S_i = 0\}$. What is its distribution?
- Q2 **Winning fraction:** What is the probability $P(\alpha, \beta)$ (for $0 \leq \alpha < \beta \leq 1$) that player A is winning between fractions α and β of the time?
- Q3 **Oscillations:** How do the fluctuations of $(S_n)_{0 \leq n \leq 2N}$ grow?
- Q4 **Dominance:** What is the probability that one player is *always* winning?

1.2.1 Key Quantities of the SSRW I

Last time at zero

$$L_{2n} = \sup\{m \leq 2n : S_m = 0\}$$

Steps spent above zero

$$L^+_{2n} = \#\{j = 1, \ldots, 2n : S_{j-1} \geq 0 \text{ or } S_j \geq 0\}$$

Total time player A is strictly winning

$$W_{2N} = \#\{1 \leq k \leq 2N : S_k > 0\}$$

1.2.1 Key Quantities of the SSRW II

Surprising fact: Intuitively, by symmetry $L_{2n}^{+}/(2n) \approx 1/2$. The **Arcsine Law** shows the opposite: the limiting density is U-shaped, with most mass near 0 and 1. The walk tends to spend *almost all* its time on one side.

Monte Carlo estimate of the distribution of $L_{2n}^{+}/(2n)$

Run R independent replications, record $\left(L_{2n,(r)}^{+}/(2n)\right)_{r=1}^{R}$, and plot a histogram. Compare to the arcsine density $p(x) = (\pi\sqrt{x(1-x)})^{-1}$.

1.2.1 Python: Simulating the SSRW (Listing 1.1)

Listing 2: Simple symmetric random walk simulation

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.random import default_rng

rng = default_rng(seed=31415)
N   = 1000

# +1 or -1 with equal probability
steps = 1 - 2 * rng.integers(0, high=2, size=N)
S     = np.concatenate([[0], steps.cumsum()])

plt.figure(figsize=(9, 4))
plt.plot(S, lw=0.8)
plt.axhline(0, color='k', lw=0.5, ls='--')
plt.xlabel('Step $n$')
plt.ylabel('$S_n$')
plt.title(f'Simple Symmetric Random Walk, $N={N}$')
plt.tight_layout()
plt.show()
```

A sample realization for $N = 1000$ is shown in Book Fig. 1.1.

1.2.1 Law of the Iterated Logarithm I

Law of the Iterated Logarithm (LIL)

$$\mathbb{P}\left(\liminf_{n\to\infty} \frac{S_n}{\sqrt{2n\log\log n}} = -1\right) \;=\; \mathbb{P}\left(\limsup_{n\to\infty} \frac{S_n}{\sqrt{2n\log\log n}} = +1\right) = 1.$$

Source: [100]; cf. Chapter VIII.5 in Feller [52].

Three scales for S_n:

Scale	Result	Verdict
S_n/n	$\to 0$ (LLN)	too strong
$S_n/\sqrt{n}$	oscillates, amplitude $\to \infty$	too weak
$S_n/\sqrt{2n\log\log n}$	$\limsup = +1$, $\liminf = -1$ a.s.	**exact**

1.2.1 Law of the Iterated Logarithm II

Book Fig. 1.2: 500 independent trajectories of length 2^{30}. The paths stay within the LIL envelope $\pm\sqrt{2n \log\log n}$ (blue band) and touch it infinitely often; the red curves show $\pm\sqrt{n}$.

Simulation insight: Even for very long walks, the envelope is never permanently exceeded — but the walk *will* return to it eventually (almost surely).

1.2.1 Python: LIL Envelope Visualization

Listing 3: Plot random walk trajectories with LIL envelope

```
import numpy as np
import matplotlib.pyplot as plt

rng, R, N = np.random.default_rng(0), 50, 2**18
n_arr    = np.arange(1, N + 1)
envelope = np.sqrt(2 * n_arr * np.log(np.log(n_arr + 2)))

plt.figure(figsize=(10, 4))
for _ in range(R):
    steps = 1 - 2 * rng.integers(0, 2, N)
    S = np.concatenate([[0], steps.cumsum()])
    plt.plot(S, lw=0.3, alpha=0.4, color='gray')

plt.plot(n_arr,  envelope, 'b-', lw=2, label=r'$\pm\sqrt{2n\log\log n}$')
plt.plot(n_arr, -envelope, 'b-', lw=2)
plt.plot(n_arr,  np.sqrt(n_arr), 'r--', lw=1, label=r'$\pm\sqrt{n}$')
plt.plot(n_arr, -np.sqrt(n_arr), 'r--', lw=1)
plt.xlabel('Step $n$')
plt.ylabel('$S_n$')
plt.title(f'LIL envelope -- {R} trajectories, $N=2^{{18}}$')
plt.legend()
plt.tight_layout()
plt.show()
```

Arcsine Law (Feller [52] p. 82; Durrett [43] p. 198)

For $0 \leq \alpha < \beta \leq 1$:

$$\mathbb{P}\left(\alpha \leq \frac{L_{2n}^{+}}{2n} \leq \beta\right) \xrightarrow{n\to\infty} \int_{\alpha}^{\beta} \frac{1}{\pi\sqrt{x(1-x)}}\, dx. \tag{1.2.1}$$

A similar result holds for L_{2n}; see Durrett [43] p. 197.

Why "arcsine"?

$$\int_{0}^{\beta} \frac{dx}{\pi\sqrt{x(1-x)}} = \frac{2}{\pi}\arcsin\left(\sqrt{\beta}\right).$$

1.2.1 The Arcsine Law II

Shape of the arcsine density:

$$p(x) = \frac{1}{\pi\sqrt{x(1-x)}}, \quad x \in (0,1).$$

- **U-shaped**: density $\to \infty$ at both $x = 0$ and $x = 1$.
- **Meaning:** $L_{2n}^{+}/(2n)$ concentrates near 0 or 1, not near $1/2$.
- **Counterintuitive:** the walk spends most of its time *entirely above* or *entirely below* zero.

Book Fig. 1.3: Histograms of $L_{2n}^{+}/(2n)$ from $R = 10^4$ replications with $n = 500$, overlaid with the arcsine density. Empirical histogram closely matches theory.

1.2.1 Python: Verifying the Arcsine Law

Listing 4: Arcsine law: simulation vs theory

```python
import numpy as np
import matplotlib.pyplot as plt

def fraction_positive(n, R=10_000, seed=0):
    rng   = np.random.default_rng(seed)
    steps = 1 - 2 * rng.integers(0, 2, (R, 2 * n))
    S     = np.hstack([np.zeros((R, 1)), steps.cumsum(axis=1)])
    # L+: step j counted if S_{j-1} >= 0 OR S_j >= 0
    above = (S[:, :-1] >= 0) | (S[:, 1:] >= 0)
    return above.sum(axis=1) / (2 * n)

fracs = fraction_positive(n=500)

x   = np.linspace(0.01, 0.99, 400)
pdf = 1.0 / (np.pi * np.sqrt(x * (1 - x)))

plt.figure(figsize=(7, 4))
plt.hist(fracs, bins=60, density=True, alpha=0.6, label='Simulated')
plt.plot(x, pdf, 'r-', lw=2, label='Arcsine density')
plt.xlabel(r'$L^+_{2n}/(2n)$')
plt.ylabel('Density')
plt.legend()
plt.title('Arcsine Law: simulation vs theory ($n=500$, $R=10^4$)')
plt.tight_layout()
plt.show()
```

1.2.2 Travelling Salesman Problem I

Problem statement

Given n cities with pairwise distances $d(c_i, c_j)$, find the permutation π^* that minimises the total tour length:

$$\pi^* = \arg\min_{\pi \in S_n} \sum_{i=1}^{n} d\big(c_{\pi(i)},\, c_{\pi(i+1)}\big), \quad c_{\pi(n+1)} \coloneqq c_{\pi(1)}.$$

Computational hardness:

- **NP-complete**: no polynomial-time algorithm is known.
- Search space: $n!$ tours. $n = 10$: 3.6×10^6; $n = 20$: 2.4×10^{18}.
- Exact methods feasible only for $n \lesssim 50$.

MC heuristics (explored in later chapters):

- Random restart: sample many permutations, keep the shortest.
- 2-opt local search with random restarts.
- **Simulated Annealing** (Ch. 7): accept worse solutions probabilistically to escape local optima.

1.2.2 Travelling Salesman Problem II

- **Cross-Entropy method** (Chs. 5, 7): fit and resample.

Book Fig. 1.4: 115 USA cities from the TSPLIB benchmark. MC-based heuristics find near-optimal tours in seconds.

1.2.2 Python: Random Tour Search for TSP

Listing 5: Baseline random search for TSP

```python
import numpy as np

def tour_length(cities, perm):
    shifted = np.roll(perm, -1)
    return np.linalg.norm(cities[perm] - cities[shifted], axis=1).sum()

def random_tour_search(cities, R=10_000, seed=42):
    rng = np.random.default_rng(seed)
    n   = len(cities)
    best_len, best_tour = np.inf, None
    for _ in range(R):
        perm = rng.permutation(n)
        L    = tour_length(cities, perm)
        if L < best_len:
            best_len, best_tour = L, perm.copy()
    return best_tour, best_len

rng    = np.random.default_rng(0)
cities = rng.random((20, 2))        # 20 random 2-D cities
tour, length = random_tour_search(cities, R=5_000)
print(f"Best random tour length (R=5000): {length:.4f}")
# This is a weak baseline; SA and CE methods (Ch.7) do much better.
```

1.2.3 Computing Integrals with Monte Carlo I

Goal: Estimate $I = \mathbb{E}[f(\mathbf{U})]$ where $\mathbf{U} \sim \text{Uniform}[0,1]^d$:

$$I = \int_{[0,1]^d} f(\mathbf{u})\, d\mathbf{u}.$$

Monte Carlo Estimator

Let $\mathbf{U}_1, \ldots, \mathbf{U}_R \overset{\text{i.i.d.}}{\sim} \text{Uniform}[0,1]^d$. Define $Z_i = f(\mathbf{U}_i)$. Then:

$$\hat{I}_R = \frac{1}{R} \sum_{i=1}^{R} Z_i \xrightarrow{R \to \infty} I \quad \text{(SLLN)}.$$

1.2.3 Computing Integrals with Monte Carlo II

Variance

$$\mathrm{Var}(\hat{I}_R) = \frac{\sigma^2}{R}, \qquad \sigma^2 = \mathrm{Var}(Z_1) = \mathrm{Var}(f(\mathbf{U})).$$

Chebyshev bound: $\mathbb{P}(|\hat{I}_R - I| > \varepsilon) \leq \sigma^2/(R\varepsilon^2)$.

Central Limit Theorem for MC

$$\sqrt{R}\,(\hat{I}_R - I) \xrightarrow{d} \mathcal{N}(0, \sigma^2).$$

RMSE $= \sigma/\sqrt{R} = \mathcal{O}(R^{-1/2})$, *regardless of the dimension* d.

Dimension-free convergence:

Method	Error	Cost ($\varepsilon = 10^{-2}$, $d = 10$)
Grid (n^d pts)	$\mathcal{O}(n^{-2/d})$	$\sim 10^{20}$
MC	$\mathcal{O}(R^{-1/2})$	$\sim 10^{4}$

For $d \geq 5$, MC is typically preferred over grid quadrature.

1.2.3 Computing Integrals with Monte Carlo IV

Discrepancy: The key quantity controlling QMC error:

$$D(\mathbf{x}_1, \ldots, \mathbf{x}_n) = \sup_{B \subseteq [0,1]^d} \left| \frac{\#\{i : \mathbf{x}_i \in B\}}{n} - \lambda(B) \right|,$$

where the sup is over all axis-aligned boxes and $\lambda(B)$ is the volume.

- For i.i.d. uniform: $D \approx \mathcal{O}(n^{-1/2})$.
- For low-discrepancy sequences: $D = \mathcal{O}((\log n)^d / n)$.

Koksma–Hlawka inequality:

$$\left| \hat{I}_n - I \right| \;\leq\; V(f) \cdot D(\mathbf{x}_1, \ldots, \mathbf{x}_n),$$

where $V(f)$ is the Hardy–Krause variation of f.

1.2.3 Python: Estimating π with MC

Listing 6: Classic pi estimation via MC

```python
import numpy as np

def estimate_pi(R, seed=0):
    rng    = np.random.default_rng(seed)
    U      = rng.random((R, 2))          # uniform on [0,1]^2
    inside = (U**2).sum(axis=1) <= 1.0   # inside unit quarter-circle
    return 4.0 * inside.mean()           # area = pi/4 -> multiply by 4

for R in [100, 1_000, 10_000, 100_000, 1_000_000]:
    pi_hat = estimate_pi(R)
    error  = abs(pi_hat - np.pi)
    se     = np.sqrt(pi_hat * (4.0 - pi_hat) / R)
    print(f"R={R:>9,}  pi_hat={pi_hat:.5f}  "
          f"|error|={error:.5f}  SE={se:.5f}")
# Expected: error ~ sigma/sqrt(R), shrinks 10x per 100x more samples
```

Convergence rate

Every $10\times$ increase in R reduces the error by a factor of $\approx \sqrt{10} \approx 3.16$.

1.3 Quasi-Monte Carlo (QMC) Methods I

Idea

Replace i.i.d. random points with a **deterministic, well-spread** sequence that covers $[0,1]^d$ more uniformly than random sampling.

Low-discrepancy sequences

Several constructive sequences achieve:

$$D(\mathbf{x}_1, \ldots, \mathbf{x}_n) \;\leq\; C_d \, \frac{(\log n)^d}{n} + \mathcal{O}\!\left(\frac{(\log n)^{d-1}}{n} \right).$$

Available in Python via `scipy.stats.qmc`:

- **Halton** Van der Corput sequences with different prime bases per dimension. Simple; mild correlations for large primes.
- **Sobol'** Constructed via bit operations. Excellent uniformity in $d \geq 2$; best for $n = 2^k$.

1.3 Quasi-Monte Carlo (QMC) Methods II

Faure Permuted base-p sequences. Proven theoretical bounds.

LatinHypercube Stratified: each row/column of the grid has exactly one sample. Good but not strictly low-discrepancy.

Book Fig. 1.5: $n = 1024$ PCG64 points (left) vs Sobol' (right) in $[0, 1]^2$. The QMC points are visibly more uniform.

1.3 Quasi-Monte Carlo (QMC) Methods III

Golden-ratio (Steinhaus) sequence – 1D

$$x_n = \{n\alpha\}, \qquad \alpha = \frac{\sqrt{5}-1}{2} \approx 0.618 \quad \text{(golden ratio)},$$

where $\{x\}$ denotes the fractional part of x. α solves $z^2 + z - 1 = 0$ and has the continued-fraction expansion $\alpha = \cfrac{1}{1 + \cfrac{1}{1 + \cdots}}$. This makes α the "most irrational" number, ensuring maximal spread of $\{n\alpha\}$ across $[0, 1)$.

Kronecker sequences

$x_n = \{n\beta\}$ has low discrepancy whenever the partial quotients in $\beta = 1/(b_1 + 1/(b_2 + \cdots))$ are bounded, $b_j \leq M$. Multidimensional Kronecker sequences $(\{n\beta_1\}, \ldots, \{n\beta_d\})$ are believed to have low discrepancy (open problem), supported by numerical evidence.

1.3 Quasi-Monte Carlo (QMC) Methods IV

QMC vs MC error rate comparison:

Method	Error	When it wins
MC	$\mathcal{O}(R^{-1/2})$	High d, non-smooth f
QMC	$\mathcal{O}((\log R)^d/R)$	Smooth f, moderate d

Caveat: The constant C_d grows with d, so for very large d the QMC advantage may not appear unless n is huge.

1.3 Python: QMC Sequences with `scipy.stats.qmc`

Listing 7: Comparing MC and QMC for pi estimation

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats.qmc import Sobol, Halton, LatinHypercube

def pi_from(U):
    return 4.0 * ((U**2).sum(1) <= 1).mean()

Rs      = [2**k for k in range(5, 15)]   # 32 ... 16384
mc_err  = []
sob_err = []
hal_err = []

for R in Rs:
    rng = np.random.default_rng(0)
    mc_err.append( abs(pi_from(rng.random((R, 2))) - np.pi))
    sob_err.append(abs(pi_from(Sobol(2, scramble=True, seed=0).random(R)) - np.pi))
    hal_err.append(abs(pi_from(Halton(2, scramble=True, seed=0).random(R)) - np.pi))

plt.loglog(Rs, mc_err,  'b-o', label='MC (PCG64)')
plt.loglog(Rs, sob_err, 'r-s', label="Sobol'")
plt.loglog(Rs, hal_err, 'g-^', label='Halton')
ref = [Rs[0]**0.5 / r**0.5 * mc_err[0] for r in Rs]
plt.loglog(Rs, ref, 'k--', label=r'$O(R^{-1/2})$')
plt.xlabel('R'); plt.ylabel('|error|')
plt.grid(True); plt.legend()
```

1.4 MC vs Quasi-MC: Comparison Examples I

Remark 1.4.2 – Discrepancy in multidimensions

Grouping consecutive numbers from a 1-D low-discrepancy sequence into d-dimensional vectors **does not** preserve the low-discrepancy property. Each dimension of the d-vector must be constructed simultaneously (as done by `scipy.stats.qmc`).

Estimating π (book p. 10): With $R = 2^{10}$ Sobol' points the error is typically 10–$50\times$ smaller than MC. The advantage is largest for smooth integrands and moderate d.

Example 1.4.2 – Winning Probability in a 2-D Game

Grid: $\{0, \ldots, N_1\} \times \{0, \ldots, N_2\}$. Starting from (i_1, i_2):

- Any coordinate hits $0 \Rightarrow$ **LOSE**.
- Both coordinates reach $N_j \Rightarrow$ **WIN**.

Transitions from (i_1, i_2):

$$(i_1, i_2) \to \begin{cases} (i_1 + 1, i_2) & \text{prob. } p_1(i_1), \\ (i_1 - 1, i_2) & \text{prob. } q_1(i_1), \\ (i_1, i_2 + 1) & \text{prob. } p_2(i_2), \\ (i_1, i_2 - 1) & \text{prob. } q_2(i_1), \\ (i_2, i_2) & \text{prob. } 1 - (p_1 + q_1 + p_2 + q_2). \end{cases}$$

Exact winning probability (Formula 1.4.1)

$$\rho(i_1, i_2) \;=\; \frac{\displaystyle\prod_{j=1}^{2}\left(\sum_{n_j=1}^{i_j}\prod_{r=1}^{n_j-1}\frac{q_j(r)}{p_j(r)}\right)}{\displaystyle\prod_{j=1}^{2}\left(\sum_{n_j=1}^{N_j}\prod_{r=1}^{n_j-1}\frac{q_j(r)}{p_j(r)}\right)},$$

where empty products equal 1.

Symmetric example: $N_1 = N_2 = 4$, $q_j(i_j) = p_j(i_j) = \frac{1}{4}$ for all j, i_j. Formula gives:

$$\rho(2,3) = \frac{3}{8} = 0.375.$$

1.4 MC vs Quasi-MC: Comparison Examples IV

Simulation algorithm – one game from (i_1, i_2):
Use two independent uniforms $U_1, U_2 \in [0, 1)$ at each step:

1. Decide *which coordinate* changes with U_1:
   - Change coord. 1 if $U_1 \in [0,\ p_1(i_1) + q_1(i_1))$;
   - Change coord. 2 if $U_1 \in [p_1 + q_1,\ p_1 + q_1 + p_2 + q_2)$;
   - No change otherwise.
2. Decide *direction* with U_2:
   - Coord. 1 up if $U_2 < p_1(i_1)/(p_1(i_1) + q_1(i_1))$, else down.
   - Coord. 2 up if $U_2 < p_2(i_2)/(p_2(i_2) + q_2(i_1))$, else down.
3. Stop if any coord hits 0 (LOSE) or all coords reach N_j (WIN).

Estimator: $\hat{Y}_R^{A_i} = \frac{1}{R}\sum_{j=1}^{R} Y_j^{A_i}$, where $Y_j^{A_i} \in \{0, 1\}$ and $A_1 =$ PCG64, $A_2 =$ LatHyp, $A_3 =$ Sobol, $A_4 =$ Halton.

1.4 MC vs Quasi-MC: Comparison Examples V

Book Table 1.2 – estimates of $\rho(2,3) = 0.375$

Avg. steps	PCG64	LatHyp	Sobol	Halton
1 000	0.3760	0.3686	0.3715	0.3750
5 000	0.3748	0.3762	0.3748	0.3750

Observations (Book Fig. 1.7):

- All four generators converge to 0.375.
- Sobol and Halton are more accurate for small R.
- The error curves for QMC are smoother (less variance).

1.4 MC vs Quasi-MC: Comparison Examples VI

Structured points – Book Fig. 1.9: Scatter-plots of $(U_1^{(i)}, U_1^{(i+1)})$ for Sobol/Halton reveal clear *linear structure* ("stratification lines"), unlike the uniform scatter of PCG64. This can cause artifacts for integrands aligned with these patterns.

Practical rule of thumb

Use **QMC** for smooth, low-dimensional integrands. For $d \gtrsim 10$, non-smooth f, or Markov-chain-based algorithms, use plain **MC**.

1.4 Python: 2-D Game Simulation – MC vs QMC

Listing 8: Winning probability: MC vs Sobol estimation

```
import numpy as np
from scipy.stats.qmc import Sobol

N1, N2, p_val = 4, 4, 0.25

def play_one_game(stream):
    """stream: iterator yielding (U1, U2) pairs."""
    i1, i2 = 2, 3
    for U1, U2 in stream:
        thr1 = 2 * p_val                  # p1 + q1
        thr2 = thr1 + 2 * p_val           # + p2 + q2
        if U1 < thr1:
            i1 += 1 if U2 < 0.5 else -1
        elif U1 < thr2:
            i2 += 1 if U2 < 0.5 else -1
        if i1 == 0 or i2 == 0:  return 0
        if i1 == N1 and i2 == N2: return 1
    return 0

def mc_estimate(R, seed=0):
    rng  = np.random.default_rng(seed)
    wins = sum(play_one_game(iter(rng.random((500, 2)))) for _ in range(R))
    return wins / R

print(f"MC  (R=2000): {mc_estimate(2000):.4f}")
```

1.5 Bibliographical Notes I

Origins of Monte Carlo:

- Developed at Los Alamos in the 1940s by von Neumann, Ulam, and Metropolis for nuclear weapons simulation.
- Named after the Monte Carlo casino (Metropolis & Ulam, 1949).

Pseudorandom number generation:

- **Knuth [75]**, Vol. 2, Ch. 3: comprehensive classical reference.
- L'Ecuyer [82]; Niederreiter [101].

Random walks and arcsine law:

- **Feller [52]**, Chapters III, XIV: arcsine law, ballot problem.
- **Durrett [43]**: modern probability, random walk results.
- Law of the iterated logarithm: [100].

Quasi-Monte Carlo:

- **Niederreiter [101]**: *Random Number Generation and Quasi-Monte Carlo Methods* — foundational monograph.

1.5 Bibliographical Notes II

- Halton [60]; Sobol' [119]; Faure [50].
- Tezuka [124]: scrambling and randomisation.
- **Glasserman [55]**: QMC in finance.
- Python: `scipy.stats.qmc` (SciPy $\geq$ 1.7).

Travelling Salesman Problem:

- **Applegate et al. [4]**: *The Traveling Salesman Problem*.
- Simulated annealing: Kirkpatrick, Gelatt & Vecchi [74] (1983).

Chapter 1 Summary I

Core concepts introduced

1. **Monte Carlo estimator**: $\hat{I}_R = \frac{1}{R}\sum_{i=1}^{R} f(U_i)$, RMSE $= \mathcal{O}(R^{-1/2})$, *dimension-free*.
2. **SSRW** $S_n = \sum_{j=1}^{n} X_j$:
   - *LIL*: fluctuations scale as $\pm\sqrt{2n \log\log n}$.
   - *Arcsine law*: $L_{2n}^{+}/(2n)$ has U-shaped arcsine distribution, not uniform on $[0, 1]$.
3. **TSP**: NP-hard; MC heuristics (SA, CE) find near-optimal tours efficiently.
4. **Quasi-MC**: low-discrepancy sequences (Sobol', Halton, golden-ratio) achieve $\mathcal{O}((\log R)^d/R)$ error for smooth f.
5. **MC vs QMC**: QMC wins for smooth, low-d problems; MC is more robust for high-d, discontinuous, or Markov-chain applications.

Chapter 1 Summary II

Key Python tools:

- `numpy.random.default_rng(seed)` — reproducible PRNG
- `rng.random(n)`, `rng.integers(0, 2, n)` — sampling
- `scipy.stats.qmc.Sobol`, `.Halton`, `.LatinHypercube` — QMC sequences

Looking ahead:

- **Chapter 2:** How are pseudorandom numbers constructed, and how do we test their quality?
- **Chapter 3:** How to sample from arbitrary distributions (not just Uniform)?
- **Chapter 4:** Rigorous statistical analysis of simulation output – confidence intervals and error estimation.